

DEVALIVE

System Architecture Document (SAD)

Version: 1.0

Prepared By: Tanmay Bonde

Project Type: SaaS Developer Monitoring Platform

Document Status: Approved

1. Introduction

1.1 Purpose

This document defines the complete system architecture of DevAlive.

It describes:

- System components
- Service interactions
- Data flow
- Monitoring workflow
- Notification workflow
- Frontend architecture
- Backend architecture
- External integrations

The objective of this document is to provide a single technical blueprint that ensures all future development remains consistent and aligned.

2. Architecture Overview

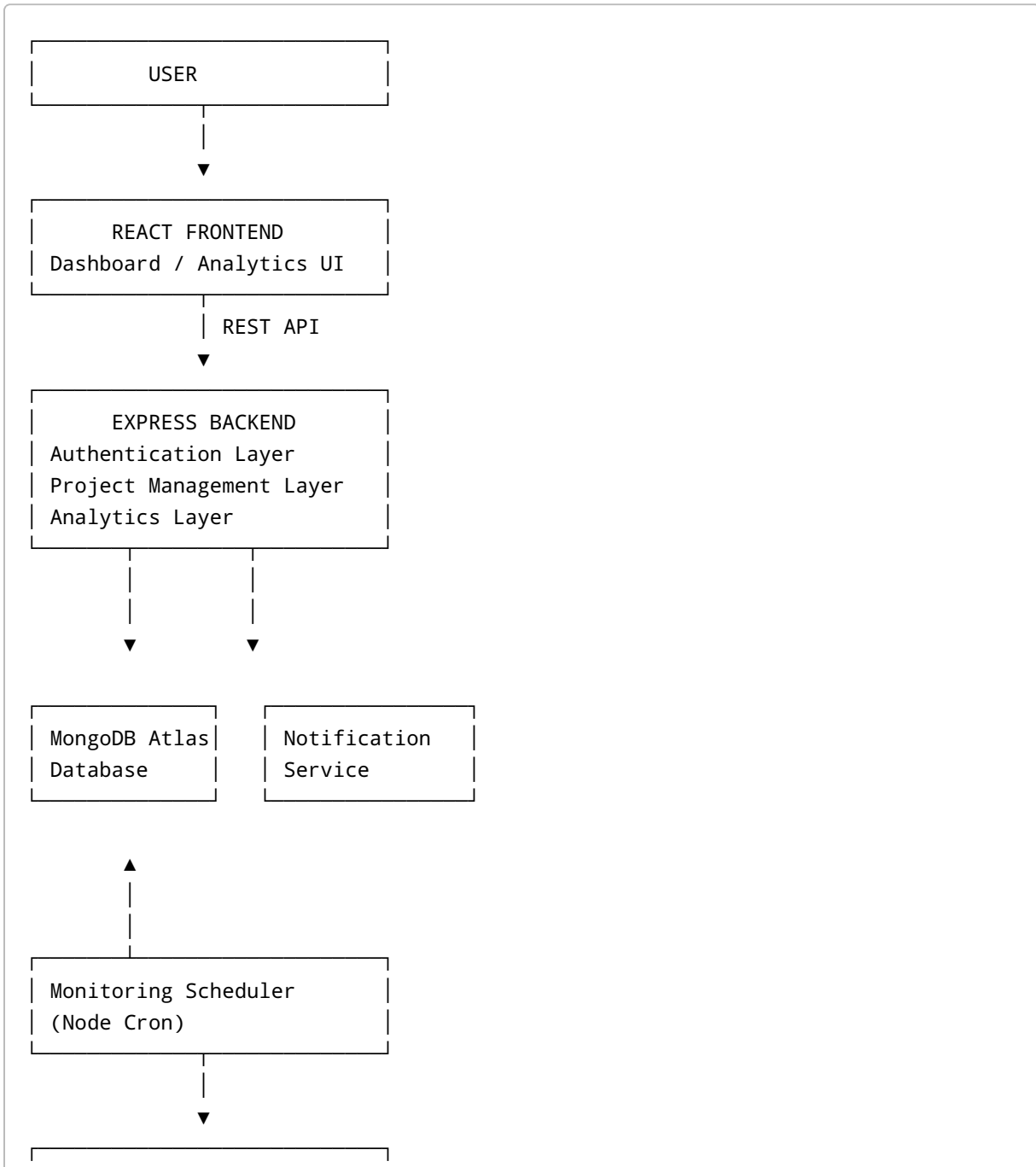
DevAlive follows a modern SaaS architecture consisting of:

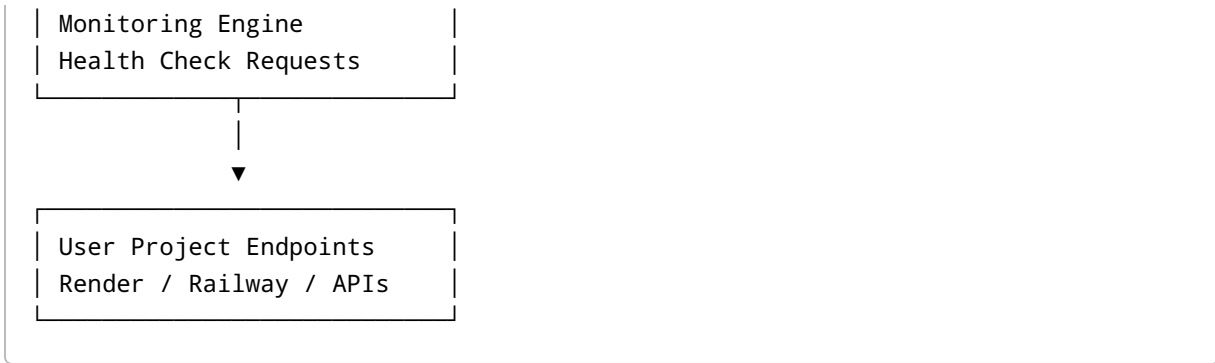
- React Frontend
- REST API Backend
- MongoDB Database
- Background Monitoring Engine

- Scheduler Service
- Notification Service

The platform operates as a centralized monitoring system that periodically checks registered project endpoints and records their availability status.

3. High Level Architecture





4. Architectural Style

DevAlive follows:

Frontend

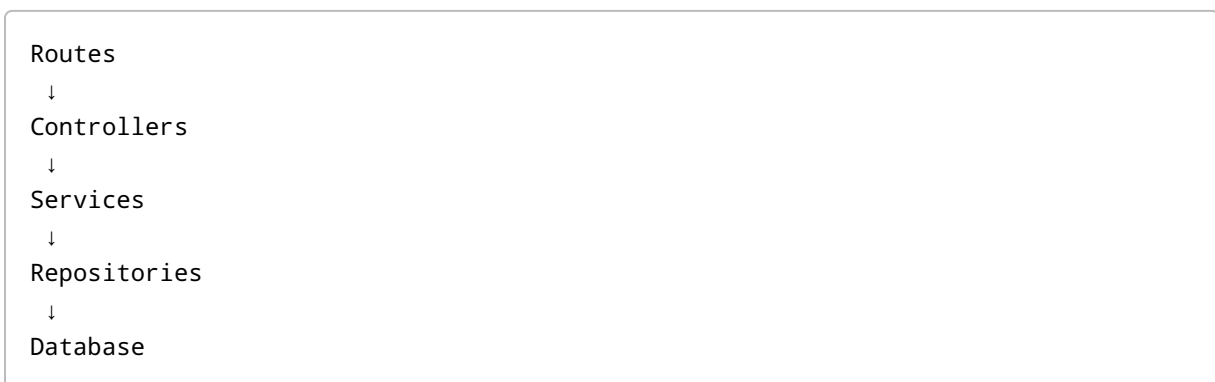
Component-Based Architecture

Technology:

- React
- Vite
- Tailwind CSS

Backend

Layered Architecture



This provides:

- Scalability

- Maintainability
 - Testability
 - Clean Code Structure
-

5. Core System Components

Component 1

Frontend Client Application

Purpose:

Provides the user interface.

Responsibilities:

- User Authentication
- Dashboard
- Project Management
- Analytics
- Notifications
- Settings

Technology:

- React
 - Vite
 - Tailwind CSS
 - Axios
 - React Router DOM
-

Component 2

Backend API Server

Purpose:

Acts as the central business logic layer.

Responsibilities:

- Authentication
- Authorization
- Project Management
- Monitoring Management
- Analytics Processing
- Notification Triggering

Technology:

- Node.js
 - Express.js
-

Component 3

Database Layer

Purpose:

Persistent data storage.

Technology:

MongoDB Atlas

Stores:

- Users
 - Projects
 - Monitoring Logs
 - Notifications
 - Settings
-

Component 4

Monitoring Scheduler

Purpose:

Automatically triggers monitoring operations.

Technology:

Node Cron

Responsibilities:

- Execute monitoring cycles
- Schedule checks
- Manage monitoring frequency

Runs continuously while backend server is active.

Component 5

Monitoring Engine

Purpose:

Core business module of DevAlive.

Responsibilities:

- Send health-check requests
- Measure response time
- Detect failures
- Detect downtime
- Update project status
- Generate logs

This is the heart of the platform.

Component 6

Notification Service

Purpose:

Send user alerts.

Responsibilities:

- Downtime alerts

- Failure alerts
- Recovery alerts
- Account notifications

Technology:

- Nodemailer
 - SMTP
-

6. System Modules

Module 1

Authentication Module

Features:

- Register
- Login
- Logout
- JWT Generation
- Password Encryption

Dependencies:

- User Collection
 - JWT Service
-

Module 2

Project Management Module

Features:

- Create Project
- Update Project
- Delete Project
- Enable Monitoring
- Disable Monitoring

Dependencies:

- Projects Collection
-

Module 3

Monitoring Module

Features:

- Scheduled Checks
- Endpoint Verification
- Status Validation
- Retry Mechanism

Dependencies:

- Monitoring Engine
 - Scheduler
-

Module 4

Analytics Module

Features:

- Uptime Calculation
- Downtime Tracking
- Monitoring History
- Dashboard Statistics

Dependencies:

- Monitoring Logs
-

Module 5

Notification Module

Features:

- Alert Generation

- Email Delivery
- Notification History

Dependencies:

- Monitoring Results
-

7. System Workflow

User Registration Flow

```
graph TD;
    User --> RegistrationForm[Registration Form];
    RegistrationForm --> BackendValidation[Backend Validation];
    BackendValidation --> PasswordHashing[Password Hashing];
    PasswordHashing --> MongoDB[MongoDB];
    MongoDB --> AccountCreated[Account Created];
```

User Login Flow

```
graph TD;
    User --> LoginForm[Login Form];
    LoginForm --> CredentialVerification[Credential Verification];
    CredentialVerification --> JWTGeneration[JWT Generation];
    JWTGeneration --> AccessGranted[Access Granted];
```

Add Project Flow

```
User
↓
Add Project Form
↓
Backend Validation
↓
Store Project
↓
Activate Monitoring
```

Monitoring Workflow

```
Scheduler Trigger
↓
Fetch Active Projects
↓
Send Health Check Request
↓
Receive Response
↓
Analyze Result
↓
Store Monitoring Log
↓
Update Project Status
```

Failure Detection Workflow

```
Health Check
↓
Failure Detected
↓
Create Failure Record
↓
Update Status
↓
Trigger Notification
```

↓
Send Email Alert

Recovery Workflow

Failed Project
↓
Subsequent Successful Check
↓
Status Updated
↓
Recovery Event Logged
↓
Recovery Notification Sent

8. Data Flow Architecture

User Data Flow

Frontend
↓
API
↓
Validation
↓
Database

Monitoring Data Flow

Scheduler
↓
Monitoring Engine
↓
Endpoint Request
↓
Response Analysis
↓

```
Monitoring Logs
↓
Analytics Engine
↓
Dashboard
```

Notification Data Flow

```
Monitoring Engine
↓
Failure Event
↓
Notification Service
↓
SMTP Provider
↓
User Email
```

9. External Integrations

MongoDB Atlas

Purpose:

Database Storage

SMTP Provider

Purpose:

Email Delivery

Examples:

- Gmail SMTP
 - SendGrid
 - Mailtrap
-

Monitored Platforms

Supported Platforms:

- Render
 - Railway
 - Vercel
 - Netlify
 - Firebase
 - Supabase
 - MongoDB Atlas APIs
 - Custom APIs
 - Public Websites
-

10. Security Architecture

Authentication:

- JWT

Password Protection:

- Bcrypt

Route Protection:

- Protected Middleware

Input Validation:

- Express Validator

Environment Variables:

- Dotenv

Security Headers:

- Helmet

Rate Limiting:

- Express Rate Limit
-

11. Scalability Architecture

Current MVP:

Single Server Architecture

```
graph TD; Frontend --> Backend; Backend --> MongoDB;
```

Future SaaS Architecture

```
graph TD; Frontend --> LB[Load Balancer]; LB --> APIS[Multiple API Servers]; APIS --> Redis[Redis Queue]; Redis --> MW[Monitoring Workers]; MW --> MongoDB[MongoDB Cluster];
```

Future Ready Features:

- Horizontal Scaling
- Queue Processing
- Distributed Monitoring
- Enterprise Accounts

12. Fault Tolerance Strategy

Monitoring Failure

Action:

Retry Request

SMTP Failure

Action:

Log Failure Retry Later

Database Failure

Action:

Return Graceful Error

Endpoint Timeout

Action:

Mark Failure Retry Based On Rules

13. Logging Strategy

Application Logs

Purpose:

Debugging

Monitoring Logs

Purpose:

Project Monitoring History

Error Logs

Purpose:

Issue Investigation

Notification Logs

Purpose:

Email Delivery Tracking

14. Architecture Principles

DevAlive architecture follows:

1. Scalability First
 2. Modular Design
 3. Security By Default
 4. Clean Separation Of Concerns
 5. SaaS-Oriented Structure
 6. Future Enterprise Readiness
 7. API-First Design
 8. Developer-Friendly Maintainability
-

15. Architecture Summary

DevAlive is designed as a modern SaaS monitoring platform consisting of:

- React Frontend
- Express Backend
- MongoDB Database
- Node Cron Scheduler
- Monitoring Engine
- Email Notification Service

The architecture prioritizes reliability, scalability, maintainability, and future expansion while remaining lightweight enough for an MVP implementation.